AGENT POSTMORTEM GENERATOR ENGINEERING PRINCIPLES

Project Constitution for a Local-First AI Infrastructure Tool

Version 1.0 | Purpose: Establish enduring engineering rules for architecture, module design, testing, documentation, and contribution standards.

This document is normative. Future decisions must comply with it unless explicitly superseded by a later revision.

1. Purpose of this Document

This document defines the enduring engineering principles for Agent Postmortem Generator. Its role is to constrain future implementation so the project remains trustworthy, maintainable, and easy to reason about even as the codebase grows.

The document exists to prevent architecture drift. It establishes the rules that govern new modules, refactors, integrations, and releases. When there is a conflict between convenience and the principles in this document, the principles win.

2. Core Engineering Philosophy

The project is built on the assumption that AI agent failures are a systems problem, not a guesswork problem. The tool must convert traces into evidence-backed narratives, not probabilistic storytelling.

The engineering posture is simple: deterministic analysis first, heuristics only when explicitly bounded, and no hidden magic. The system should be understandable by inspection and debuggable with standard software engineering tools.

The tool should feel like infrastructure, not an AI demo. That means predictable behavior, narrow responsibilities, strong contracts, and outputs that engineers can verify.

3. Architectural Principles

Local-first execution is mandatory. The default workflow must work entirely on a developer laptop without network access, external APIs, or remote state.

Normalization is the only place where framework-specific knowledge belongs. After traces are normalized, downstream analysis must operate on a project-owned canonical model.

Each layer must have a single reason to change. Parsing, normalization, analysis, reporting, and export must remain separate concerns.

Dependencies must be controlled and boring. Prefer small, stable libraries over broad platform dependencies that can destabilize the toolchain.

4. Software Design Principles

Favor simplicity over cleverness. A boring implementation that engineers can maintain beats a sophisticated one that only the original author can explain.

Prefer explicit data flow over implicit side effects. Inputs, transformation steps, and outputs should be visible in module interfaces and test fixtures.

Keep cohesion high and coupling low. Modules should own one concept, not a vague bundle of related behaviors.

Every new abstraction must earn its place by removing real duplication or isolating a true variability point.

5. Data Modeling Principles

The canonical trace model is the contract of the project. It must be strict enough to support stable analysis and flexible enough to represent multiple frameworks after normalization.

Raw inputs and normalized data must be distinct. Raw trace data should be preserved for provenance, while normalized records should power analysis.

Data models should encode meaning, not UI convenience. If a field is only useful for presentation, keep it in the reporting layer rather than polluting the core model.

6. API Design Principles

APIs should be explicit, typed where practical, and designed around stable domain concepts. Callers should never need to know internal implementation details to use them correctly.

Public interfaces must be minimal. Each exposed function, CLI flag, or config option becomes a compatibility promise and therefore needs restraint.

Behavior must be predictable across identical inputs. Input determinism is a requirement, not a nice-to-have.

7. Module Boundary Rules

Modules communicate through stable data contracts, not shared mutable state. Cross-module imports should be deliberate, not casual.

Framework adapters may depend on framework-specific libraries, but core analysis modules must not depend on any single framework's object model.

Rendering and export must stay downstream of analysis. Reporting layers may format conclusions, but they must not reinterpret the evidence.

8. Error Handling Philosophy

Failures must be classified, not hand-waved. The tool should distinguish malformed input, unsupported input, partial data, and internal defects.

Error messages must help the user act. A good error names the problem, identifies the failing stage, and points to remediation or fallback behavior.

Silent degradation is only acceptable when explicitly documented and when it does not compromise trust in the final report.

9. Logging Philosophy

Logs are for developers and operators, not for the final report. They should explain what happened inside the tool, not duplicate the incident narrative.

Logging must support debugging without drowning users in noise. Important transitions, validation failures, and summary metrics deserve visibility; routine internal chatter does not.

Logs must never become part of the analysis source of truth.

10. Testing Philosophy

Tests are part of the product, not a side quest. The project should rely on fixture-driven tests that lock in normalized behavior and report structure.

The most important tests are deterministic golden tests over canonical traces and their expected postmortem outputs.

Framework adapters require contract tests. Every supported adapter must prove it normalizes into the same canonical model for equivalent underlying events.

11. Documentation Standards

Documentation must explain architecture, data flow, module ownership, and extension points. If a contributor cannot understand where a change belongs, the docs are insufficient.

Every public module should have a concise purpose statement, input/output description, and failure modes.

Docs should be maintained as a first-class artifact. Stale documentation is technical debt that actively harms trust.

12. Repository Standards

The repository should make the architecture obvious at a glance. Top-level directories must correspond to meaningful layers or deliverables, not arbitrary historical accidents.

Generated artifacts, temporary files, and one-off experiments must not pollute the main tree.

Repository structure should support fast contributor orientation: core, adapters, tests, docs, examples, and release assets should be easy to locate.

13. Naming Conventions

Names should describe domain intent, not implementation mechanics. A reader should infer purpose from the name without reading three files.

Avoid overloaded terms. If the project defines a canonical trace, a normalized event, and a report section, each term should remain distinct.

Stable naming matters because the project is meant to outlive any single implementation generation.

14. Performance Philosophy

The default expectation is fast local analysis on ordinary developer hardware. Excessive latency is a product defect, not an optimization opportunity.

Performance work must be evidence-led. Do not pre-optimize. Measure first, then fix bottlenecks that actually matter to the workflow.

Determinism and correctness outrank micro-optimizations. A fast wrong answer is worse than a slightly slower trusted one.

15. Extensibility Rules

Extensibility must be designed as explicit plugin or adapter boundaries, not as a pile of optional branches inside the core engine.

New integrations must conform to the canonical model instead of forcing the model to mirror every upstream framework's quirks.

Extending the system should not require rewriting existing modules. If it does, the abstraction boundary is wrong.

16. Open Source Contribution Standards

Contributions must preserve readability, tests, and documentation quality. A feature that cannot be explained or tested is not ready to merge.

Changes to public contracts require explicit review, migration notes, and compatibility impact assessment.

Contributor experience matters. The project should welcome outside engineers without forcing them to reverse-engineer the architecture.

17. Decision-Making Framework

Architectural decisions should be made by answering four questions: does it preserve determinism, does it improve trust, does it reduce complexity, and does it increase long-term maintainability?

The default answer to new complexity is no. Complexity must justify itself with a specific, durable benefit.

When tradeoffs are unavoidable, prefer the choice that keeps the canonical model stable and the analysis pipeline inspectable.

18. Anti-Patterns to Avoid

Do not bury framework-specific logic inside analysis code. That creates brittle coupling and makes future adapters painful.

Do not let LLM output become the source of truth for the postmortem. It may assist presentation later, but evidence must drive conclusions.

Do not create a giant omniscient service object. It makes the system look simple until it becomes impossible to change.

Do not normalize with ad hoc rules scattered across the codebase. All normalization logic must be centralized and testable.

19. Examples of Good vs Bad Design Decisions

20. Checklist for Reviewing Future Modules

19. Examples of Good vs Bad Design Decisions

The point of these examples is not style preference. It is to show which decisions protect the architecture and which decisions quietly rot it from the inside.

Decision	Recommended approach	Why it matters
Good	Parse raw traces into a canonical model, then analyze only the normalized representation.	Keeps the analysis pipeline framework agnostic and testable.
Bad	Let every framework-specific trace shape leak directly into the report generator.	Creates duplicated logic and guarantees future adapter pain.
Good	Return structured error types for invalid traces, unsupported formats, and internal failures.	Lets callers handle failures precisely and preserves trust.
Bad	Return one generic exception for every problem.	Makes debugging slower and hides the difference between user error and system defects.
Good	Use fixture-based golden tests for representative failure traces.	Locks in deterministic behavior and prevents accidental regressions.
Bad	Rely on manual testing with a few happy-path examples.	Leaves core failure handling unverified and fragile.

20. Checklist for Reviewing Future Modules

- ☐ Does the module have a single, clearly named responsibility?
- ☐ Does it operate on the canonical normalized model rather than raw framework objects?
- ☐ Are inputs, outputs, and error cases explicit?
- ☐ Does it preserve determinism for identical inputs?
- ☐ Are tests deterministic, fixture-driven, and meaningful?

☐ Does it avoid hidden side effects and global state?

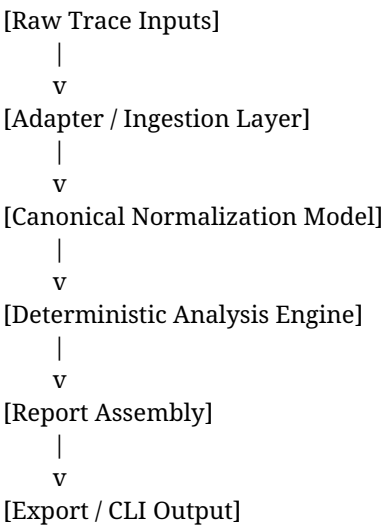
☐ Is its failure mode visible to callers and users?

☐ Does it keep framework-specific knowledge at the adapter boundary?

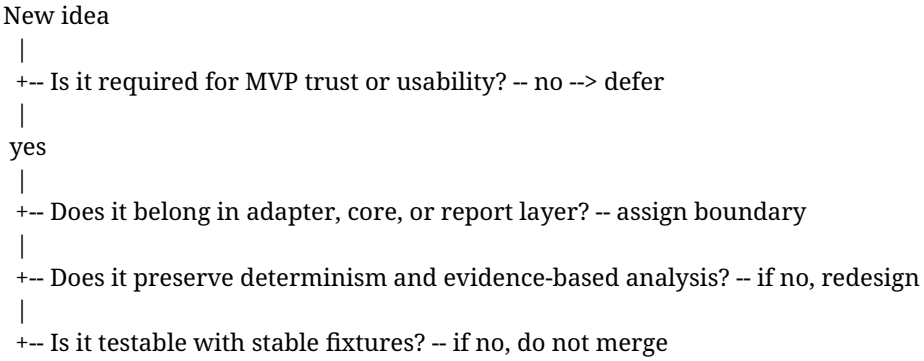
☐ Is the module documented with purpose, contract, and limitations?

☐ Would this module still make sense one or two years from now?

System Boundary Reference



Decision Flow for New Work



Architectural Principles

Principle	Why it exists	Problem it prevents	How it should influence future work
Local-first execution	The tool must be usable without remote services.	Prevents latency, privacy concerns, and operational fragility.	Future work must preserve offline operation as the default path.

Canonical normalization	Downstream analysis needs one stable data shape.	Prevents adapter logic from contaminating every later stage.	Any new integration must map into the canonical model before analysis.
Deterministic pipeline	Trust requires repeatable results.	Prevents output drift from hidden state or probabilistic dependence.	If behavior changes, the reason must be visible in code or input.

Software Design Principles

Principle	Why it exists	Problem it prevents	How it should influence future work
Single responsibility	Smaller modules are easier to reason about and test.	Prevents mixed concerns that become impossible to refactor safely.	Every module proposal should have one primary job.
Explicit contracts	Callers need to understand exactly what a component accepts and returns.	Prevents accidental coupling and implicit behavior.	Public interfaces should stay narrow and documented.
Low coupling, high cohesion	Well-bound modules scale better as the project grows.	Prevents cross-cutting edits that touch unrelated files.	Any shared logic should live at the correct abstraction level.

Data Modeling Principles

Principle	Why it exists	Problem it prevents	How it should influence future work
Separate raw and normalized data	Provenance matters and so does stable analysis.	Prevents trace loss and accidental mutation of original	Keep raw inputs immutable and preserved for

		evidence.	debugging.
Evidence-bearing fields only	The model should describe what happened, not what we wish happened.	Prevents speculative or decorative schema growth.	Add fields only when they support analysis or validation.
Stable semantics	A field should mean the same thing across adapters.	Prevents incompatible interpretations between frameworks.	Schema changes require compatibility review.

API Design Principles

Principle	Why it exists	Problem it prevents	How it should influence future work
Minimal public surface	Public APIs become obligations.	Prevents a bloated support burden and accidental lock-in.	Expose only what the project can defend long-term.
Predictable behavior	Developers need confidence that inputs map to stable outcomes.	Prevents surprising edge cases and hard-to-debug regressions.	If behavior is conditional, document the conditions.
Typed or structured interfaces	Structure reduces ambiguity.	Prevents stringly-typed glue code and hidden assumptions.	Favor explicit objects or records over ad hoc blobs.

Error Handling Philosophy

Principle	Why it exists	Problem it prevents	How it should influence future work
Classify failures	Users must know what kind of failure occurred.	Prevents generic exceptions that hide actionable detail.	Return or raise distinct error classes when possible.

Actionable messages	Errors should help fix the problem.	Prevents support churn and fruitless debugging.	Each error message should mention stage, cause, and remedy.
No silent corruption	Bad data should not become confident nonsense.	Prevents false trust in incorrect reports.	If recovery happens, record it clearly.

Testing Philosophy

Principle	**Why it exists**	**Problem it prevents**	**How it should influence future work**
Golden fixtures	Core outputs must remain stable over time.	Prevents accidental drift in normalized analysis and report generation.	Use representative failure traces and expected outputs.
Contract tests for adapters	Adapters are boundary objects.	Prevents adapter regressions from breaking the canonical model.	Every supported source format needs explicit coverage.
Deterministic tests only	Flaky tests destroy trust in the suite.	Prevents false negatives and ignored test results.	Eliminate time, randomness, and network dependency from tests.